



Concurso Internacional de Programação Colegial 2020

Concursos regionais da América Latina

10 de julho de 2021

Sessão do concurso

Este conjunto de problemas contém 14 problemas; as páginas são numeradas de 1 a 28.

Esse conjunto de problemas é usado em concursos simultâneos realizados nos seguintes países:

Argentina, Bolívia, Brasil, Chile, Colômbia, Costa Rica, Cuba, Equador, El Salvador, Guatemala, Jamaica, México, Nicarágua, Porto Rico, República Dominicana, Trinidad e Tobago e Venezuela

Informações gerais

Salvo indicação em contrário, as condições a seguir são válidas para todos os problemas.

Nome do programa

1. Sua solução deve se chamar *codename.c*, *codename.cpp*, *codename.java*, *codename.kt*, *code-name.py3*, em que *codename* é a letra maiúscula que identifica o problema.

Entrada

1. A entrada deve ser lida a partir da entrada padrão.
2. A entrada consiste em um único caso de teste, que é descrito usando um número de linhas que depende do problema. Nenhum dado extra aparece na entrada.
3. Quando uma linha de dados contém vários valores, eles são separados por espaços *simples*. Nenhum outro espaço aparece na entrada. Não há linhas vazias.
4. É usado o alfabeto inglês. Não há letras com tildes, acentos, diaereses ou outros sinais diacríticos (*n*~, *A*~, 'e', 'I', ^o, 'U'', ,c, etc.).
5. Cada linha, inclusive a última, tem a marca de fim de linha usual.

Saída

1. A saída deve ser gravada na saída padrão.
2. O resultado do caso de teste deve aparecer na saída usando um número de linhas que depende do problema. Nenhum dado extra deve aparecer na saída.
3. Quando uma linha de resultados contém vários valores, eles devem ser separados por espaços *simples*. Nenhum outro espaço deve aparecer na saída. Não deve haver linhas vazias.
4. O alfabeto inglês deve ser usado. Não deve haver letras com tildes, acentos, diaereses ou outros sinais diacríticos (*n*~, *A*~, 'e', 'I', ^o, 'U'', ,c, etc.).
5. Cada linha, inclusive a última, deve ter a marca de fim de linha usual.
6. Para gerar números reais, arredonde-os para o racional mais próximo com o número necessário de dígitos após o ponto decimal. O caso de teste é tal que não há empates ao arredondar conforme especificado.

Problema A -Quase Origami

Autor : Alejandro Strejilevich de Loma, Argentina

Você tem uma folha de papel retangular de altura 1 e deseja localizar qualquer ponto na altura H medida a partir da borda inferior da folha. Como você não conhece os teoremas de Haga, planeja repetir a etapa a seguir. Suponha que você já tenha localizado um ponto P_L na altura L , na borda esquerda da folha, e um ponto P_R na altura R , na borda direita da folha. Em seguida, você desenha uma linha do canto inferior esquerdo da planilha até P_R , e outra linha do canto inferior direito da planilha até P_L . Se o ponto de cruzamento estiver na altura H , você terá terminado. Caso contrário, você desenhara uma linha horizontal que passe pelo ponto de cruzamento e seguirá para outra etapa.

Como exemplo, considere o caso de $H = 1/3$. Durante a primeira etapa, a única possibilidade é escolher os cantos superiores da folha (ou seja, $L = R = 1$). Portanto, você desenha as duas diagonais da folha, e o ponto de cruzamento está na altura $1/2$. Como $H \neq 1/2$, você desenha uma linha horizontal que passa pelo ponto de cruzamento. Essa linha fornece dois novos pontos com altura conhecida de $1/2$ nas bordas da folha, um na borda esquerda e o outro na borda direita. Na segunda etapa, você pode escolher entre usar os pontos conhecidos originais na altura 1 ou os pontos que acabou de localizar na altura $1/2$. Ou seja, você pode escolher entre $L = 1$ ou $L = 1/2$ e, é claro, $R = 1$ ou $R = 1/2$. É fácil ver que, se você escolher $L = R = 1/2$, o ponto de cruzamento estará na altura $1/4$. Entretanto, se você escolher $L = 1/2$ e $R = 1$, o ponto de cruzamento estará na altura desejada $H = 1/3$. Por simetria, o mesmo ocorre se você escolher $L = 1$ e $R = 1/2$.

Dada uma altura racional H , você deve determinar a sequência mais curta de alturas nas bordas da folha que permita localizar um ponto na altura H .

Como mostra o exemplo acima, somente um ponto na altura $1/2$ pode ser localizado em uma única etapa e, portanto, uma possível sequência mais curta para $H = 1/3$ é $S = (1, 1, 1/2, 1)$. As duas primeiras alturas devem ser escolhidas durante a primeira etapa, e as duas alturas restantes devem ser escolhidas durante a segunda etapa.

Entrada

A entrada consiste em uma única linha que contém dois números inteiros M e N ($1 \leq M < N \leq 100$), de modo que $H = M/N$ é uma fração irredutível.

Saída

Emita uma única linha com o caractere "*" (asterisco) se um ponto na altura H não puder ser localizado por meio do procedimento descrito. Caso contrário, produza a sequência mais curta de alturas $S_1, S_2, \dots, S_K$ que permite localizar um ponto na altura H , se forem escolhidas na ordem em que aparecem na sequência. A altura S_i deve ser escrita na i -ésima linha usando dois números inteiros A_i e B_i , de modo que $S_i = A_i/B_i$ seja uma fração irredutível ($i = 1, 2, \dots, K$). É garantido que, quando um ponto na altura H puder ser localizado, ele poderá ser localizado de forma ideal escolhendo apenas alturas racionais.

Entrada de amostra 1 1 3	Saída de amostra 1 1 1 1 1 1 2 1 1
Entrada de amostra 2 1 3	Exemplo de saída 2 1 1 1 1 1 1 1 2
Entrada de amostra 3 3 4	Exemplo de saída 3 *

Entrada de amostra 4	Exemplo de saída 4
1 4	1 1 1 1 1 2 1 2

Problema B -Belas montanhas

Autor : Andre Amaral de Sousa, Brasil

Uma submatriz de uma matriz é uma parte contígua da matriz. Uma partição de uma matriz em subvariedades é uma coleção de subvariedades que cobrem toda a matriz sem sobreposições (cada elemento da matriz pertence exatamente a uma subvariedade). Por exemplo, se $A = [3, 1, 4, 1, 5]$ for uma matriz, $[3, 1, 4]$ e $[1, 5]$ formam uma partição de A em sub matrizes, enquanto $[3, 4, 5]$ *não* é uma sub matriz de A .

Essas são definições padrão que você pode ter lido em outros lugares. Então, o que há de novo aqui? Bem, a seguir, mais algumas definições.

Dada uma matriz A de números inteiros, uma submatriz $[A_i, A_{i+1}, \dots, A_j]$ de A é chamada de montanha se existir um índice k de tal forma que $i < k < j$, a submatriz de A_i a A_k não é decrescente e a submatriz de A_k a A_j não é crescente. Em palavras simples, os valores na subvariedade "sobem" até o índice k e, em seguida, "descem", parecendo uma montanha. Observe que uma subvariedade com menos de três elementos não pode ser uma montanha.

Uma matriz de números inteiros é chamada de cadeia de montanhas bonitas se puder ser dividida em montanhas, cada uma delas com o mesmo número de elementos, exceto a última montanha, que pode ter menos elementos.

Por exemplo, $[5, 10, 4, 1, 3, 2]$ é uma bela cadeia de montanhas porque pode ser dividida em $[5, 10, 4]$ e $[1, 3, 2]$, sendo que ambas as montanhas têm o mesmo número de elementos. Outro exemplo é a matriz $[5, 10, 4, 4, 10, 20, 30, 20, 2, 3, 1]$, que também é uma bela cadeia de montanhas porque pode ser divididos em $[5, 10, 4, 4]$, $[10, 20, 30, 20]$ e $[2, 3, 1]$.

Dada uma matriz de números inteiros positivos, em que alguns valores podem estar faltando, determine se é possível completar a matriz com números inteiros positivos de modo que ela se torne uma bela cadeia de montanhas.

Entrada

A primeira linha contém um número inteiro N ($3 \leq N \leq 10^5$) indicando o número de elementos na matriz. A segunda linha contém N inteiros $A_1, A_2, \dots, A_N$ ($A_i = -1$ ou $1 \leq A_i \leq 10^9$ para $i = 1, 2, \dots, N$), em que $A_i = -1$ indica que o i -ésimo elemento da matriz precisa ser determinado, e um valor positivo é o i -ésimo elemento real da matriz.

Saída

Emita uma única linha com a letra maiúscula "Y" se for possível completar a matriz com números inteiros positivos, de modo que ela se torne uma bela cadeia de montanhas, e a letra maiúscula "N" caso contrário.

Entrada de amostra 1 6 5 10 4 1 3 2	Saída de amostra 1 Y
Entrada de amostra 2 11 5 10 4 -1 10 20 30 20 2 3 -1	Exemplo de saída 2 Y
Entrada de amostra 3 12 1 3 2 5 -1 8 9 -1 7 -1 4 5	Exemplo de saída 3 N

Esta página seria intencionalmente deixada em branco se não quiséssemos informar sobre isso.

Problema C -Crise no casamento

Autor : Ricardo Anido, Brasil

Um famoso jogador de futebol acabou de se casar e está organizando uma festa para os convidados do casamento. Os convidados estão sentados em mesas ao redor de um lago circular no jardim da casa do jogador. Cada mesa acomoda exatamente o mesmo número de convidados, e as mesas consecutivas ao redor do lago estão a uma distância unitária.

No momento do tradicional brinde do padrinho, surgiu uma crise: embora o número total de taças de champanhe nas mesas dos convidados seja exatamente o número de convidados, as taças poderiam ter sido distribuídas de forma desigual entre as mesas, com algumas mesas com mais taças do que convidados e outras com menos taças do que convidados.

Um único garçom está disponível para consertar a distribuição de copos, coletando os copos excedentes das mesas e entregando-os às mesas que precisam de copos. O custo de cada reparo de copo é a distância que o garçom carrega o copo até entregá-lo a uma mesa. O custo total da operação é a soma dos custos de todos os copos. O garçom pode começar em qualquer mesa, mas o jogador é supersticioso e só permitirá que o garçom caminhe estritamente no sentido horário ou anti-horário ao fixar a distribuição dos copos. Ou seja, quando o garçom começa em uma direção (sentido horário ou anti-horário), ele não pode mudar a direção.

Ganhe uma camisa autografada do jogador de futebol americano ajudando-o a calcular o menor custo total possível para consertar a distribuição de óculos.

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 10^5$) indicando o número de mesas ao redor do lago circular. A segunda linha contém N números inteiros $G_1, G_2, \dots, G_N$ ($0 \leq G_i \leq 1000$ para $i = 1, 2, \dots, N$), representando o número de copos nas diferentes mesas. Esses números são fornecidos em ordem horária.

É garantido que N divide $\sum_{i=1}^N G_i$.

Saída

Emita uma única linha com um número inteiro indicando o menor custo total possível para fixar a distribuição de óculos.

Entrada de amostra 1 4 14 10 6 10	Saída de amostra 1 8
Entrada de amostra 2 6 24 122 0 37 49 242	Exemplo de saída 2 454
Entrada de amostra 3 6 0 0 0 0 60 0	Exemplo de saída 3 150
Entrada de amostra 4 1 0	Exemplo de saída 4 0

Esta página seria intencionalmente deixada em branco se não quiséssemos informar sobre isso.

Problema D -Dividindo o doce

Autor : Arthur Nascimento, Brasil

Bob e Charlie são dois irmãos que gostam muito de poderes de 2. Sua mãe decidiu dar a eles N caixas de doces, cada uma delas contendo um número de barras de chocolate que é uma potência de 2.

Eles querem dividir as caixas entre si, ou seja, para cada caixa, eles decidirão quem ficará com ela. Cada caixa deve ser dada a exatamente um irmão.

Agora eles se perguntam: é possível que, para cada um dos dois irmãos, a quantidade total de barras de chocolate que ele recebe também seja uma potência de 2?

Por exemplo, se $N = 4$ e as caixas contêm 4, 4, 32 e 8 barras de chocolate, a resposta seria *sim*, pois uma solução possível é dar a terceira caixa para Bob (32 barras de chocolate) e as caixas restantes para Charlie ($4 + 4 + 8 = 16$ barras de chocolate no total).

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 10^5$), o número de caixas que os irmãos desejam dividir. A segunda linha contém N números inteiros $A_1, A_2, \dots, A_N$ ($0 \leq A_i \leq 10^5$ para $i = 1, 2, \dots, N$), indicando que a i -ésima caixa tem 2^{A_i} barras de chocolate.

Saída

Emita uma única linha com a letra maiúscula "Y" se for possível dividir as caixas de modo que a quantidade total de doces recebidos por cada irmão seja uma potência de 2 e a letra maiúscula "N" caso contrário.

Entrada de amostra 1 4 2 2 5 3	Saída de amostra 1 Y
Entrada de amostra 2 1 42	Exemplo de saída 2 N
Entrada de amostra 3 5 3 1 4 1 5	Exemplo de saída 3 N
Entrada de amostra 4 7 0 0 1 2 3 4 5	Exemplo de saída 4 Y

Esta página seria intencionalmente deixada em branco se não quiséssemos informar sobre isso.

Problema E -Excelentes visualizações

Autor : Giovanna Kobus Conrado, Brasil

Shiny City é uma cidade linda, famosa por três coisas: o fato de ter apenas uma rua, o fato de todos os edifícios terem alturas diferentes e as vistas de tirar o fôlego do topo desses edifícios.

Desde o início da pandemia, o número de turistas que visitam Shiny City diminuiu significativamente. Você está determinado a escrever um blog incrível para atrair mais turistas e impedir a ruína financeira da sua cidade adorável, mas terrivelmente ineficiente. Infelizmente, ainda faltam algumas informações no blog.

Em Shiny City, há N edifícios, e o i -ésimo edifício é identificado por sua posição i . Ir do edifício i ao edifício j leva $|i-j|$ minutos. Cada edifício tem uma altura diferente H_i , e quanto mais alto o edifício, melhor a vista de seu topo.

Se você estiver em um determinado prédio, talvez valha a pena ir para um prédio diferente que tenha uma vista melhor. Devido aos custos de transporte, nunca vale a pena ir a um prédio se houver um mais alto que você possa alcançar sem gastar mais tempo.

Formalmente, podemos dizer que ir do edifício i para outro edifício j vale a pena se não houver k de modo que $|i-k| \leq |i-j|$ e $H_j < H_k$. Observe que k pode ser igual a i .

Você quer escrever em seu blog, para cada edifício, quantos outros edifícios valem a pena ir a partir dele. Por favor, reúna essas informações, caso contrário, Shiny City estará condenada para sempre.

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 10^5$), o número de edifícios em Shiny City. A segunda linha contém N números inteiros diferentes $H_1, H_2, \dots, H_N$ ($1 \leq H_i \leq 10^9$ para $i = 1, 2, \dots, N$), em que H_i é a altura do edifício i .

Saída

Produza uma única linha com N números inteiros, de modo que o i -ésimo deles represente o número de edifícios para os quais vale a pena ir a partir do edifício i .

Entrada de amostra 1	Saída de amostra 1
10 23 20 7 30 43 70 5 42 67 10	3 4 3 2 1 0 1 2 1 2

Esta página seria intencionalmente deixada em branco se não quiséssemos informar sobre isso.

Problema F - Partições fascinantes

Autor : Arthur Nascimento, Brasil

Uma submatriz de uma matriz é uma parte contígua da matriz. Uma partição de uma matriz em subvariedades é uma coleção de subvariedades que cobrem toda a matriz sem sobreposições (cada elemento da matriz pertence exatamente a uma subvariedade). Por exemplo, se $A = [3, 1, 4, 1, 5]$ for uma matriz, $[3, 1, 4]$ e $[1, 5]$ formam uma partição de A em sub matrizes, enquanto $[3, 4, 5]$ *não* é uma sub matriz de A .

Considerando uma matriz de números inteiros e uma partição da matriz em subvariedades não vazias, definimos o custo de cada subvariedade como seu elemento máximo e o custo de toda a partição como a soma dos custos das subvariedades.

Como exemplo, considere a matriz $[3, 5, 7, 1, 2, 4]$. A partição formada pelas sub matrizes $[3, 5]$, $[7]$ e $[1, 2, 4]$ tem custo $5 + 7 + 4 = 16$, enquanto a partição formada pelas sub matrizes $[3]$, $[5, 7, 1]$ e $[2, 4]$ tem custo $3 + 7 + 4 = 14$. Ambas as partições são formadas por $k = 3$ submatrizes, mas têm custos diferentes. Outras partições podem ter custos diferentes.

Dada uma matriz A de N números inteiros e um número inteiro k , de modo que $1 \leq k \leq N$, considere o conjunto $P(A, k)$ que contém todas as partições de A em k sub-matrizes não vazias. Você consegue calcular o custo mínimo em $P(A, k)$? Você também pode calcular o custo máximo? Para cada k possível? Certo, então vá em frente.

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 8000$), o número de elementos na matriz A . A segunda linha contém N números inteiros $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$ para $i = 1, 2, \dots, N$) representando a matriz.

Saída

Produza N linhas, de modo que a k -ésima linha contenha dois números inteiros indicando, respectivamente, os custos mínimo e máximo em $P(A, k)$.

Entrada de amostra 1 6 3 5 7 1 2 4	Saída de amostra 1 7 7 10 12 12 16 14 19 17 21 22 22
Entrada de amostra 2 4 1 1 1 1	Exemplo de saída 2 1 1 2 2 3 3 4 4

Esta página seria intencionalmente deixada em branco se não quiséssemos informar sobre isso.

Problema G Jogo de caça-níqueis

Autor : Arthur Nascimento, Brasil

Alice e Bob estão jogando o seguinte jogo. Inicialmente, cada um deles recebe N cartas, cada uma com um número inteiro escolhido de forma aleatória e uniforme no intervalo $[1, 10^{18}]$ escrito nela. Há também N espaços, numerados de 1 a N .

Alice começa colocando suas cartas nos espaços, de modo que exatamente uma carta seja colocada em cada espaço. Ela pode escolher qualquer uma das $N!$ opções de colocação que desejar. Depois disso, Bob observa os números colocados por Alice em cada espaço e faz o mesmo com suas cartas. O jogo termina e, para cada slot i , a pessoa que colocou o maior número ganha i pontos ($i = 1, 2, \dots, N$). Se ambos os números colocados forem iguais, os jogadores dividem os pontos para essa casa, ou seja, cada jogador recebe $i/2$ pontos.

Como exemplo do jogo, suponha que $N = 3$, Alice receba cartas com os números 10, 20 e 40 e Bob receba cartas com os números 10, 30 e 50. Alice pode escolher colocar os números 40, 20 e 10 nos espaços 1, 2 e 3, respectivamente. Depois de ver as jogadas dela, Bob pode optar por colocar os números 30, 50 e 10 nos espaços 1, 2 e 3, respectivamente. Nesse caso, Alice ganha 1 ponto para o espaço 1, 0 pontos para o espaço 2 e $3/2 = 1,5$ pontos para o espaço 3. Sua pontuação total é de 2,5 pontos.

O objetivo do exemplo acima é ilustrar as regras do jogo. Alice e Bob podem não colocar suas cartas conforme descrito, pois ambos jogam de forma otimizada. Você pode se perguntar o que isso significa.

Na sua vez, Bob tem todas as informações sobre as cartas e decisões de Alice e, é claro, ele conhece suas cartas. Ele joga de forma a maximizar sua pontuação total.

Por outro lado, Alice não sabe quais são as cartas de Bob quando chega a sua vez. Ela conhece a estratégia de Bob e sabe que as cartas dele foram escolhidas de forma aleatória e uniforme. Assim, ela joga para maximizar o valor esperado de sua pontuação total.

Dado N , você deve calcular o valor esperado da pontuação total de Alice no final do jogo, se os jogadores jogarem de forma otimizada, conforme descrito.

Entrada

A entrada consiste em uma única linha que contém um número inteiro N ($1 \leq N \leq 100$), o número de slots no jogo.

Saída

Produza uma única linha com um número que indique o valor esperado da pontuação total de Alice no final do jogo, se os jogadores jogarem de forma otimizada, conforme descrito. O resultado deve ser exibido como um número racional com exatamente seis dígitos após o ponto decimal, arredondado, se necessário.

Entrada de amostra 1	Saída de amostra 1
1	0.500000
Entrada de amostra 2	Exemplo de saída 2
2	1.333333
Entrada de amostra 3	Exemplo de saída 3
99	589.631287

Esta página seria intencionalmente deixada em branco se não quiséssemos informar sobre isso.

Problema H -Halting Wolf

Autor : Teodoro Freund, Argentina

Senoof adora linguagens de programação, e a única coisa que ele gosta mais do que usá-las é criar novas. Sua última invenção é a Wolf Programming Language, uma linguagem muito simples que consiste em apenas dois tipos de instruções. Elas são numeradas consecutivamente e escritas uma sob a outra para formar um programa. A execução começa na instrução 1 e continua até o programa *t r a v a r*.

Os dois tipos de instruções são:

- " $K L_1 L_2 \dots L_K$ " é um salto finito. Cada valor L_i é um número de instrução no programa, enquanto K indica quantos deles estão especificados. Quando um salto finito é executado, um dos valores L_i é escolhido e a execução continua com a instrução L_i . Mas isso não é tudo! O programa altera a instrução de salto finito de modo a consumir o valor escolhido. Se um programa executar um salto finito sem valores disponíveis, ele ficará preso e será interrompido.
- " $*L$ " é um salto infinito. Quando ele é executado, o programa continua com a instrução L , deixando a instrução de salto infinito inalterada.

Eu sei, Senoof é uma loucura, mas não é tão difícil. A figura abaixo mostra um exemplo, em que a instrução atual é indicada com um sinal d , e um valor consumido é denotado com um sinal H . O programa em (a) inicia a execução na instrução 1, que é um salto finito. Suponha que o segundo valor seja escolhido, ou seja, a execução continua com a instrução 2 e esse valor é consumido na instrução 1, o que produz a situação mostrada em (b). Como a instrução 2 é um salto infinito para a instrução 3, a execução continua com essa instrução, sem consumir nenhum valor da instrução 2. Agora imagine que, a partir da instrução 3, a execução salta para a instrução 4, depois para a instrução 1 e, em seguida, novamente para a instrução 1, consumindo os valores correspondentes. A situação nesse ponto é mostrada em (c). Como você pode ver, o programa fica preso e é interrompido, pois não há valores disponíveis para o salto.

d 1: 2 1 2
2: * 3
3: 3 4 3 4
4: 2 1 1

(a)

1: 2 1 H
d 2: * 3
3: 3 4 3 4
4: 2 1 1

(b)

d 1: 2 $H H$
2: * 3
3: 3 H 3 4
4: 2 H 1

(c)

Depois de algumas brincadeiras, Senoof percebeu que os programas escritos em Wolf podem ser executados para sempre, o que não significa que uma determinada instrução possa ser executada infinitas vezes. Ele gentilmente nos enviou o seguinte exemplo de um programa que pode ser executado para sempre, embora a instrução 1 possa ser executada no máximo duas vezes.

1: 2 1 2
2: * 4
3: 3 4 3 4
4: * 2

Considerando um programa escrito em Wolf, você deve determinar o número máximo de vezes que a instrução 1 pode ser executada.

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 100$), o número de instruções que o programa tem. Cada uma das próximas N linhas descreve uma instrução. Um salto finito é representado por um número inteiro não negativo K seguido por K números inteiros $L_1, L_2, \dots, L_K$ ($1 \leq L_i \leq N$ para $i = 1, 2, \dots, K$). Por outro lado, um salto infinito é descrito com o caractere "*" (asterisco) seguido de um número inteiro L ($1 \leq L \leq N$). É garantido que a quantidade total de instruções mencionadas nos saltos finitos é de no máximo 10.⁴

Saída

Emite uma única linha com um número inteiro indicando o número máximo de vezes que a instrução 1 pode ser executada, ou o caractere "*" (asterisco) se a instrução 1 puder ser executada infinitas vezes.

Entrada de amostra 1 4 2 1 2 * 3 3 4 3 4 2 1 1	Saída de amostra 1 3
Entrada de amostra 2 4 2 1 2 * 4 3 4 3 4 * 2	Exemplo de saída 2 2
Entrada de amostra 3 4 2 2 3 2 3 4 1 1 1 1	Exemplo de saída 3 3
Entrada de amostra 4 3 * 3 * 1 * 2	Exemplo de saída 4 *
Entrada de amostra 5 1 0	Exemplo de saída 5 1

Problema I - Parede impenetrável

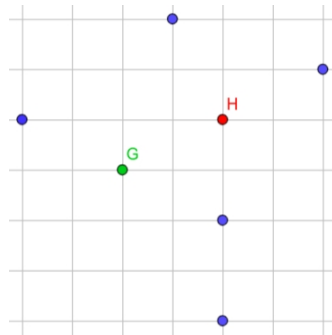
Autor : Giovanna Kobus Conrado, Brasil

A presidente de Happyland precisa ser reeleita. Ela fará isso concentrando seus últimos esforços em uma das coisas que a população de Happyland mais ama: crianças. Ela sabe que é um grande problema o fato de as crianças continuarem fugindo do orfanato nacional, por isso decidiu reconstruir o muro ao redor para que o orfanato fique mais seguro e a população mais feliz.

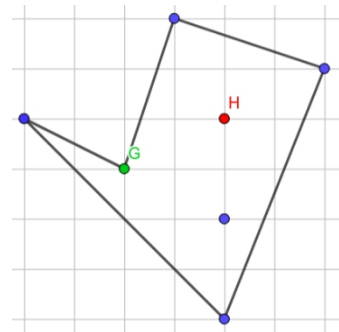
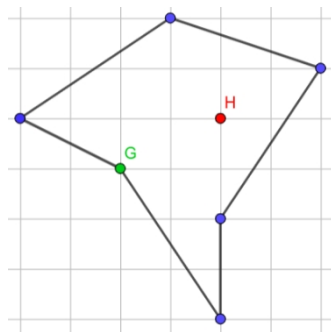
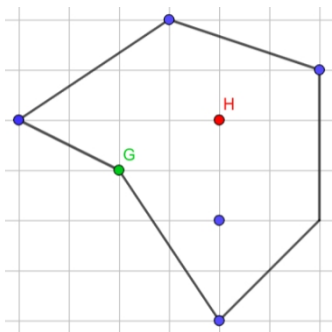
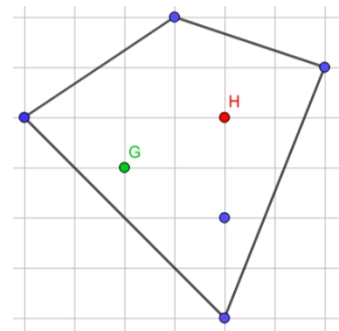
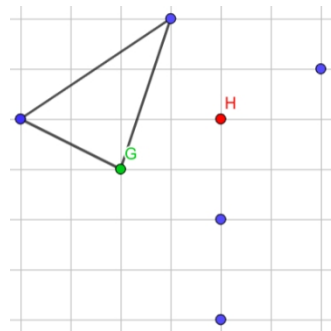
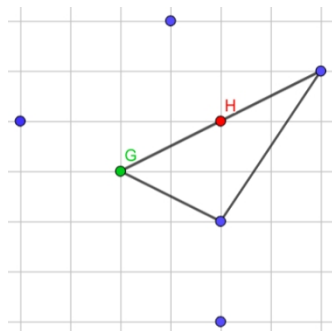
A área em torno da qual o muro deve ser construído consiste na casa do orfanato, no portão do orfanato e em um conjunto de torres de observação, remanescentes dos dias de glória de Happyland.

O presidente da Happyland determinou, em uma reunião com especialistas em orfanatos, que uma parede segura deve ser um polígono tal que:

1. A casa é estritamente cercada pelo muro.
2. O portão é um vértice da parede e todos os outros vértices são torres de observação.
3. Todos os ângulos internos dos vértices da parede que são torres de observação são estritamente menores que 180 graus (observe que isso não se aplica ao vértice do portão).
4. A parede inteira é visível a partir da casa. Isso significa que, para cada ponto da parede, o segmento entre a casa e esse ponto não passa pela parede.

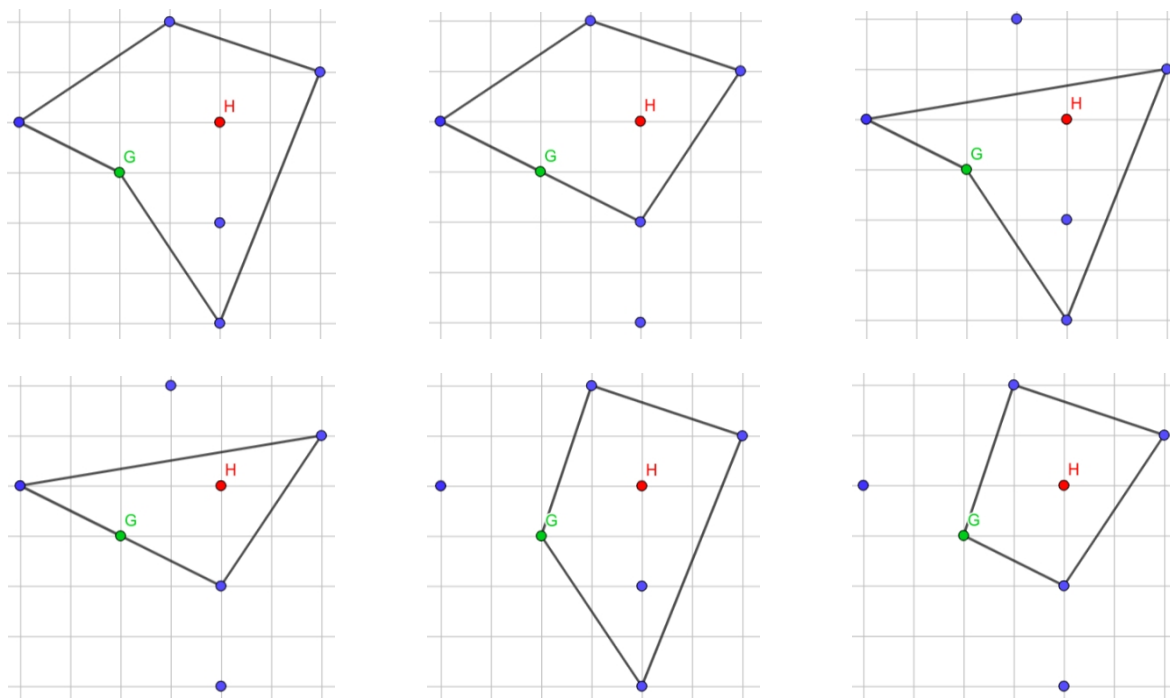


Uma possível configuração de objeto como na entrada de amostra 1. O ponto H representa a casa, o ponto G é o portão, e todos os outros pontos são torres de observação.



As paredes representadas acima violam uma ou mais regras, portanto, não são paredes válidas.

Agora o presidente quer sua ajuda para saber quantos muros seguros distintos podem ser construídos. Duas paredes são distintas se e somente se houver uma torre de observação que seja um vértice de uma parede, mas não da outra.



As paredes representadas acima são todas as paredes válidas distintas para a entrada de amostra 1.

Entrada

A primeira linha contém dois números inteiros X_h e Y_h , as coordenadas da casa. A próxima linha contém dois números inteiros X_g e Y_g , as coordenadas do portão. A linha seguinte contém um número inteiro N ($0 \leq N \leq 300$), o número de torres de observação. Cada uma das próximas N linhas contém dois números inteiros X_t e Y_t , as coordenadas de uma torre. Todas as coordenadas mencionadas (X, Y) são distintas e tais que $-10^9 \leq X, Y \leq 10^9$.

Saída

Produza uma única linha com um número inteiro indicando o número de paredes distintas que seguem as restrições. Como esse número pode ser muito grande, gere o restante da divisão por $10^9 + 7$.

Entrada de amostra 1 0 0 -2 -1 5 2 1 0 -4 -1 2 -4 0 0 -2	Saída de amostra 1 6
Entrada de amostra 2 2 2 2 0 3 -1 2 4 7 2 -2	Exemplo de saída 2 1
Entrada de amostra 3 100 100 -200 -120 0	Exemplo de saída 3 0

Problema J - Alocador de trabalhos

Autor : Gabriel Poesia, Brasil

O Consórcio de Infraestrutura para Computação Pública (ICPC) é uma rede de computadores operada por voluntários de todo o mundo que compartilham recursos de computação entre si. Os colaboradores podem conectar e desconectar suas máquinas na rede e também executar trabalhos de computação nas máquinas da rede. Com o ICPC, projetos importantes que, de outra forma, teriam custos de infraestrutura proibitivos (como a execução de juízes on-line para competições de programação) tornam-se empreendimentos viáveis.

Por mais que pareça ótimo no papel, por enquanto, o ICPC é apenas um sonho. Para que ele funcione, falta uma peça fundamental de software: o alocador de tarefas. É aqui que você entra: a comunidade está contando com você para fazer essa importante (mas voluntária, é claro) contribuição.

A rede é extremamente dinâmica: as máquinas se conectam e se desconectam o tempo todo. O alocador de tarefas precisa manter o controle das máquinas que estão conectadas no momento e quais recursos elas compartilham. Há vários tipos de recursos, como núcleos de CPU, GPUs e discos SSD. Uma máquina pode compartilhar um ou mais recursos, possivelmente mais de um do mesmo tipo. Além disso, a qualquer momento, os usuários podem solicitar máquinas para executar trabalhos de computação. Para isso, eles especificam uma lista de recursos de que uma máquina precisa para executar o trabalho, e o alocador de trabalho precisa determinar quantas das máquinas conectadas no momento têm todos os recursos necessários para executar o trabalho. Por exemplo, para um trabalho que precisa de um núcleo de CPU e duas GPUs, o alocador precisaria contar quantas máquinas têm pelo menos um núcleo de CPU e duas ou mais GPUs.

Sua tarefa é simplesmente contar quantas máquinas conectadas satisfazem os requisitos de recursos de cada trabalho, já que outro voluntário assumiu a tarefa de implementar a atribuição real de trabalhos às máquinas. Toda a comunidade do ICPC depende de você. Você pode ajudar?

Entrada

A primeira linha contém dois números inteiros N ($1 \leq N \leq 10^5$) e K ($1 \leq K \leq 8$), indicando, respectivamente, o número de eventos de rede que devem ser processados e o número de tipos de recursos disponíveis no ICPC. Os eventos são descritos em ordem cronológica nas próximas N linhas, um evento por linha. Há três tipos de eventos.

Se o evento representar que uma nova máquina está sendo conectada à rede, a linha conterá a letra maiúscula "C", seguida de um número inteiro R ($1 \leq R \leq 8$) indicando o número de recursos que a máquina está compartilhando, seguido de R inteiros $T_1, T_2, \dots, T_R$ ($1 \leq T_i \leq K$ para $i = 1, 2, \dots, R$), descrevendo o tipo de cada um dos recursos compartilhados. O ICPC atribui implicitamente identificadores inteiros sequenciais exclusivos a novas máquinas, começando em 1.

Quando o evento representa que uma máquina está sendo desconectada da rede, a linha contém a letra maiúscula "D", seguida de um número inteiro que indica o identificador da máquina. É garantido que esse identificador corresponda a uma máquina conectada válida.

Por fim, se o evento representar que um usuário deseja executar um trabalho, a linha conterá a letra maiúscula "J", seguida de um número inteiro R ($1 \leq R \leq 8$) indicando o número de recursos que o trabalho precisa, seguido de R inteiros $T_1, T_2, \dots, T_R$ ($1 \leq T_i \leq K$ para $i = 1, 2, \dots, R$), descrevendo o tipo de cada um dos recursos necessários. É garantido que a entrada contenha pelo menos um evento desse tipo.

Saída

Emite uma linha para cada evento do tipo "J". A linha deve conter um número inteiro indicando o número de máquinas que, no momento do evento, estão conectadas à rede e fornecem todos os recursos solicitados. Escreva os resultados em ordem cronológica, ou seja, usando a mesma ordem da entrada.

Entrada de amostra 1	Saída de amostra 1
3 2	1
C 3 1 1 2	0
J 2 1 2	
J 2 2 2	

Entrada de amostra 2	Exemplo de saída 2
11 3 C 2 1 2 J 1 3 C 3 1 2 3 J 1 3 J 1 1 D 2 J 1 3 J 2 1 2 J 2 1 1 D 1 J 1 2	0 1 2 0 1 0 0

Problema K - Keylogger

Autor : Cristhian Bonilha, Brasil

Ultimamente, você tem ficado muito curioso sobre sua velocidade de digitação e tem se perguntado quanto tempo leva para pressionar cada tecla do teclado, que tem teclas K .

Para descobrir isso, você instalou um keylogger em seu próprio computador. Ele vem registrando o tempo delta entre cada par de pressionamentos de tecla. Depois de coletar dados por algumas semanas, você agora tem acesso a uma matriz bidimensional T com K linhas e K colunas. O elemento na i -ésima linha e j -ésima coluna é $T_{i,j}$, e representa quanto tempo leva, em média, para você pressionar a tecla j logo após ter pressionado a tecla i . Por exemplo, o elemento $T_{3,5}$ representa quanto tempo leva, em média, para você pressionar a tecla 5 logo após ter pressionado a tecla 3. Coincidentemente, cada linha de T é ordenada de forma não decrescente.

Como a sua velocidade de digitação varia de acordo com a hora do dia e o seu humor, o keylogger também lhe deu um erro de margem de latência L . Isso significa que, para cada par de teclas i e j no seu teclado, leva entre $T_{i,j} - L$ e $T_{i,j} + L$, inclusive, para você pressionar a tecla j logo após ter pressionado a tecla i .

Você se classificou para a competição regional sul-americana do ICPC e foi solicitado a atualizar algumas de suas informações de contato no site do ICPC. O problema é que você tem estudado tanto que esqueceu sua senha. Tudo o que você lembra é que sua senha tem comprimento N . Felizmente, o keylogger também tem dados sobre a última vez que você digitou sua senha no site.

Portanto, agora você tem uma matriz P com $N - 1$ elementos. Cada elemento P_i representa o tempo delta entre cada pressionamento de tecla consecutivo de sua senha. Em outras palavras, P_1 representa o tempo delta entre o pressionamento das teclas do primeiro e do segundo caracteres da sua senha, P_2 é o tempo delta entre o pressionamento das teclas do segundo e do terceiro caracteres da sua senha, e assim por diante. Observe que a latência L não se aplica a P , porque cada P_i não é uma média, mas um único tempo delta, medido com precisão.

Você precisa recuperar sua senha o mais rápido possível. Seu plano agora é tentar todas as sequências de chaves que sejam compatíveis com as informações que você tem. Uma sequência S de comprimento N é compatível com L , T e P se cada par de chaves consecutivas S_i e S_{i+1} satisfizer que $T_{S_i, S_{i+1}} - L \leq P_i \leq T_{S_i, S_{i+1}} + L$. Quantas sequências desse tipo existem?

Entrada

A primeira linha contém dois números inteiros K ($1 \leq K \leq 750$) e L ($0 \leq L \leq 10^9$), indicando, respectivamente, quantas teclas existem no teclado e a margem de erro de latência fornecida pelo keylogger. As próximas K linhas contêm K números inteiros cada, representando a matriz T . O j -ésimo número inteiro na i -ésima linha é $T_{i,j}$ ($1 \leq T_{i,j} \leq 10^9$ para $i = 1, 2, \dots, K$ e $j = 1, 2, \dots, K$). Lembre-se de que $T_{i,j}$ indica quanto tempo leva, em média, para você pressionar a tecla j logo após ter pressionado a tecla i , e que cada linha em T é ordenada de forma não decrescente ($T_{i,j} \leq T_{i,j+1}$ para $i = 1, 2, \dots, K$ e $j = 1, 2, \dots, K - 1$). A próxima linha contém um número inteiro N ($2 \leq N \leq 10^4$), representando o tamanho de sua senha. A linha final contém $N - 1$ números inteiros $P_1, P_2, \dots, P_{N-1}$ ($1 \leq P_i \leq 10^9$ para $i = 1, 2, \dots, N - 1$), denotando o tempo delta entre cada pressionamento de tecla consecutivo de sua senha.

Saída

Produza uma única linha com um número inteiro indicando quantas sequências diferentes de chaves são compatíveis com as informações que você tem. Como esse número pode ser muito grande, produza o restante da divisão por $10^9 + 7$.

Entrada de amostra 1	Saída de amostra 1
4 0 1 1 3 5 2 4 4 10 1 1 1 8 5 6 7 8 5 2 3 8 5	1

Entrada de amostra 2 3 3 9 10 15 9 13 16 3 5 6 3 10 5	Exemplo de saída 2 0
Entrada de amostra 3 5 1 1 5 6 8 10 1 2 4 5 5 5 5 5 6 8 3 3 3 4 5 1 1 3 4 5 4 1 3 7	Exemplo de saída 3 4

Problema L -Programação da Lola

Autor : Paulo Cezar Pereira Costa, Brasil

Lola é uma menina cheia de energia e com muitos interesses, o que faz com que cada dia seja um oceano de possibilidades para ela, repleto de atividades empolgantes das quais ela está mais do que disposta a participar. Infelizmente, muitas das atividades de que Lola participa acontecem em espaços fechados e, por isso, seus níveis de vitamina D estão um pouco abaixo do ideal. Para ajudar, um médico receitou um suplemento vitamínico que ela deve tomar diariamente a cada X minutos.

Lola criou um aplicativo para acompanhar suas atividades. O principal recurso do aplicativo é o agendamento de atividades. Cada atividade consiste em um título e em seus horários de início e término. O aplicativo também permite a criação de lembretes únicos, que consistem simplesmente em um título e o momento em que ela deve receber uma única notificação, e lembretes recorrentes, que consistem em um título, o horário da primeira notificação e a frequência com que o lembrete deve ser repetido. Depois de comprar o suplemento, Lola deseja adicionar um lembrete recorrente ao aplicativo, que se repete a cada X minutos, para garantir que ela seja notificada sempre que for a hora de tomar o suplemento. Idealmente, ela prefere não ter que tomar o suplemento enquanto estiver participando de uma atividade. Com uma agenda tão ocupada, ela está tendo dificuldade em encontrar o momento ideal para começar a tomar o suplemento.

Sua tarefa é ajudar Lola a escolher o horário ideal T após a compra do suplemento para tomá-lo pela primeira vez. Um horário T é considerado ideal se for, no máximo, 8 horas após o momento em que Lola comprou o suplemento, se o número de vezes em que ela precisa tomar o suplemento que conflita com suas atividades for mínimo e se não houver outro horário anterior a T que resulte no mesmo número de conflitos.

Para fins de clareza, suponha que Lola precise tomar o suplemento a cada 30 minutos, que o tenha comprado em um determinado dia às 10h00 e que tenha uma única atividade programada das 18h00 às 18h30 durante esse dia. Se ela optar por tomar o suplemento pela primeira vez imediatamente, ela o tomaria às 10:00, 10:30, . . . Assim, ela teria dois conflitos com sua atividade (um às 18:00 e outro às 18:30).

Embora Lola não possa esperar até o término de sua atividade para tomar o suplemento pela primeira vez (devido ao limite de 8 horas), ela pode reduzir o número de conflitos esperando 7 horas e 59 minutos. Dessa forma, ela tomaria o suplemento às 17:59, 18:29, 18:59, com um único conflito às 18:29.

Muito melhor, não é? No entanto, isso não é o ideal, porque há um horário anterior que também produz um único conflito, que é esperar 1 minuto e tomar o suplemento pela primeira vez às 10h01, com um único conflito às 18h01. Como não é possível evitar completamente os conflitos, o horário ideal, nesse caso, é tomar o suplemento pela primeira vez 1 minuto depois de comprá-lo.

Você receberá uma lista de atividades, exportada do aplicativo que Lola escreveu, contendo informações sobre todas as atividades dela a partir do momento em que ela comprou o suplemento. Lola já fez algum pré-processamento dos dados e cada atividade é descrita pelo número de minutos a partir do momento em que ela comprou o suplemento e sua duração em minutos.

Entrada

A primeira linha contém dois números inteiros N ($1 \leq N \leq 10^4$) e X ($1 \leq X \leq 720$), indicando que Lola tem N atividades futuras exportadas de seu aplicativo e que ela deve tomar o suplemento a cada X minutos. Cada uma das N linhas seguintes descreve uma atividade com dois números inteiros S e D ($1 \leq S, D \leq 10^5$), representando que a atividade começa S minutos após a compra do suplemento e que sua duração é de D minutos. As atividades não se sobrepõem, ou seja, dado qualquer par de atividades diferentes, uma delas termina estritamente antes do início da outra, ou vice-versa.

Saída

Produza uma única linha com dois números inteiros T e C , indicando, respectivamente, o momento ideal para tomar o suplemento pela primeira vez, expresso em minutos desde a compra do suplemento, e o número de conflitos que a ingestão do suplemento nesse momento causará.

Entrada de amostra 1	Saída de amostra 1
1 30 480 30	1 1

Entrada de amostra 2 5 30 195 30 120 45 240 30 30 60 300 180	Exemplo de saída 2 451 1
Entrada de amostra 3 4 720 60 30 150 75 750 60 1500 60	Exemplo de saída 3 0 0
Entrada de amostra 4 2 720 1 479 482 298	Exemplo de saída 4 0 1

Problema M -Posso adicionar uma letra?

Autor : Arthur Nascimento, Brasil

Você tem uma string S de comprimento N e é solicitado a executar uma sequência de atualizações Q de dois tipos para

S :

- Acrescenta um determinado caractere ao final de S .
- Exclui o último caractere de S .

Inicialmente e após cada atualização, você deve calcular o número de cadeias distintas que ocorrem pelo menos duas vezes como subcadeias de S .

Por exemplo, se S for inicialmente "ABABC", a resposta será 3, pois "A", "B" e "AB" ocorrem duas vezes como substrings de S . Se você for solicitado a acrescentar o caractere "C", S se tornará "ABABCC" e a resposta será 4, pois agora "C" também ocorre duas vezes. Se você for solicitado a acrescentar "C" novamente, S será "ABABCCC" e a resposta será 5, pois "CC" ocorre duas vezes agora. Se você receber uma operação de exclusão agora, S se tornará "ABABCC" e a resposta será 4 novamente.

Entrada

A primeira linha contém uma string S de comprimento N ($1 \leq N \leq 10^5$), indicando o valor inicial da string. Cada caractere de S é uma letra maiúscula. A segunda linha contém uma cadeia de caracteres U de comprimento Q ($1 \leq Q \leq 10^5$), representando as atualizações a serem realizadas. Cada caractere de U é uma letra maiúscula, indicando que essa letra deve ser anexada, ou o caractere "-" (hífen), indicando uma operação de exclusão. As atualizações devem ser aplicadas na ordem em que aparecem em U . É garantido que as operações de exclusão não sejam aplicadas a cadeias de caracteres vazias.

Saída

Saída de $Q + 1$ linhas, cada linha com um número inteiro indicando o número de cadeias distintas que ocorrem pelo menos duas vezes como subcadeias de S . A linha 1 refere-se ao valor inicial de S , enquanto a linha $i + 1$ refere-se ao valor de S após a aplicação das primeiras i atualizações ($i = 1, 2, \dots, Q$).

Entrada de amostra 1 ABABC CC-	Saída de amostra 1 3 4 5 4
Entrada de amostra 2 ABAB A--CC	Exemplo de saída 2 3 5 3 1 1 2
Entrada de amostra 3 TER DIVERSÃO	Exemplo de saída 3 0 0 0 0

Esta página seria intencionalmente deixada em branco se não quiséssemos informar sobre isso.

Problema N - Donuts sem número inteiro

Autor : Paulo Cezar Pereira Costa, Brasil

Neil é um advogado muito importante com uma conta bancária muito importante. Como Neil é um advogado muito bem-sucedido e tem muitos clientes, ele deposita dinheiro em sua conta todas as manhãs.

Depois de ir ao banco e depositar o dinheiro, Neil vai para o trabalho. E lá está o grande ponto fraco de Neil: uma loja de donuts. Veja bem, Neil é um viciado em rosquinhas em recuperação e, apesar de não comer rosquinhas há anos, ele não consegue deixar de pensar em quantas rosquinhas de US\$ 1,00 ele poderia comprar com o dinheiro em sua conta se tivesse uma recaída.

Ter US\$ 5,00 em sua conta significa que Neil poderia comer 5 donuts, mas e US\$ 4,50? Bem, isso é mais do que 4 rosquinhas com certeza, mas definitivamente menos do que 5. Como alguém poderia comprar uma quantidade não inteira de rosquinhas? Esse conceito confunde Neil, então toda vez que o saldo da conta não é um número inteiro, ele para para refletir sobre a natureza dos donuts não inteiros e acaba se atrasando para o trabalho.

Agora Neil se atrasou muitas vezes e está começando a se preocupar com a possibilidade de perder o emprego. Ele quer saber quantas vezes chegará atrasado ao trabalho nos próximos N dias, considerando o saldo inicial da conta e a quantia de dinheiro que depositará a cada dia. Por favor, responda isso para ele, senão Neil começará a refletir novamente.

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 1000$), o número de dias em que Neil está interessado. Cada uma das próximas $N + 1$ linhas contém uma string que representa uma quantia de dinheiro. A primeira string é o saldo inicial da conta do Neil, enquanto as N strings seguintes são os valores que o Neil depositará em sua conta nos diferentes dias. Cada string tem o formato $\$X.Y$, em que X é uma substring de comprimento 1 ou 2 que indica o dinheiro inteiro no valor $\$X.Y$, enquanto Y é uma substring de comprimento exatamente 2 que indica os centavos no valor $\$X.Y$. Tanto X quanto Y são formados por dígitos, pelo menos um deles contém um dígito diferente de zero, e X não tem zeros à esquerda.

Saída

Emite uma única linha com um número inteiro indicando quantas vezes Neil chegará atrasado ao trabalho nos próximos N dias.

Entrada de amostra 1 1 \$1.57 \$3.14	Saída de amostra 1 1
Entrada de amostra 2 4 \$1.00 \$0.01 \$0.99 \$10.00 \$98.76	Exemplo de saída 2 2

Esta página seria intencionalmente deixada em branco se não quiséssemos informar sobre isso.